

Relatório do Projeto Final

Autor: Luiz Gustavo Turatti

Fase 1: Design e Estrutura para a Modernização do Sistema EventMaster

1. Visão Arquitetural

A solução proposta tem como objetivo transformar a atual arquitetura monolítica do sistema em um ecossistema distribuído de microsserviços. Esta modernização será fundamentada nos seguintes pilares técnicos:

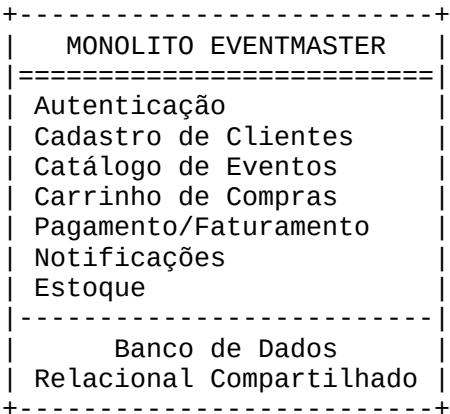
- Arquitetura Limpa (Clean Architecture)
- Design Orientado a Domínio (DDD)
- Implementação de API Gateway
- Padrão de Resiliência Circuit Breaker
- Padrão SAGA para transações distribuídas
- Adoção de práticas rigorosas de codificação segura
- Validação por meio de protótipos funcionais em Java

Atualmente, o monólito centraliza operações críticas de negócio, como autenticação , catálogo de eventos , carrinho de compras , faturamento , notificações e controle de estoque. Essa centralização gera deficiências severas, incluindo alto nível de acoplamento , intensa competição e bloqueios no banco de dados , gargalos críticos de escalabilidade , risco de propagação em cascata de falhas e exposição a vulnerabilidades de segurança. A proposta de modernização resolve esses desafios ao segregar responsabilidades em serviços autônomos.

2. Transição Estrutural — Do Monólito aos Microsserviços

Arquitetura Monolítica (Estado Atual)

O sistema EventMaster opera concentrando Autenticação, Registro de Clientes, Catálogo de Eventos, Carrinho de Compras, Pagamento/Faturamento, Notificações e Controle de Estoque em uma única base de código.



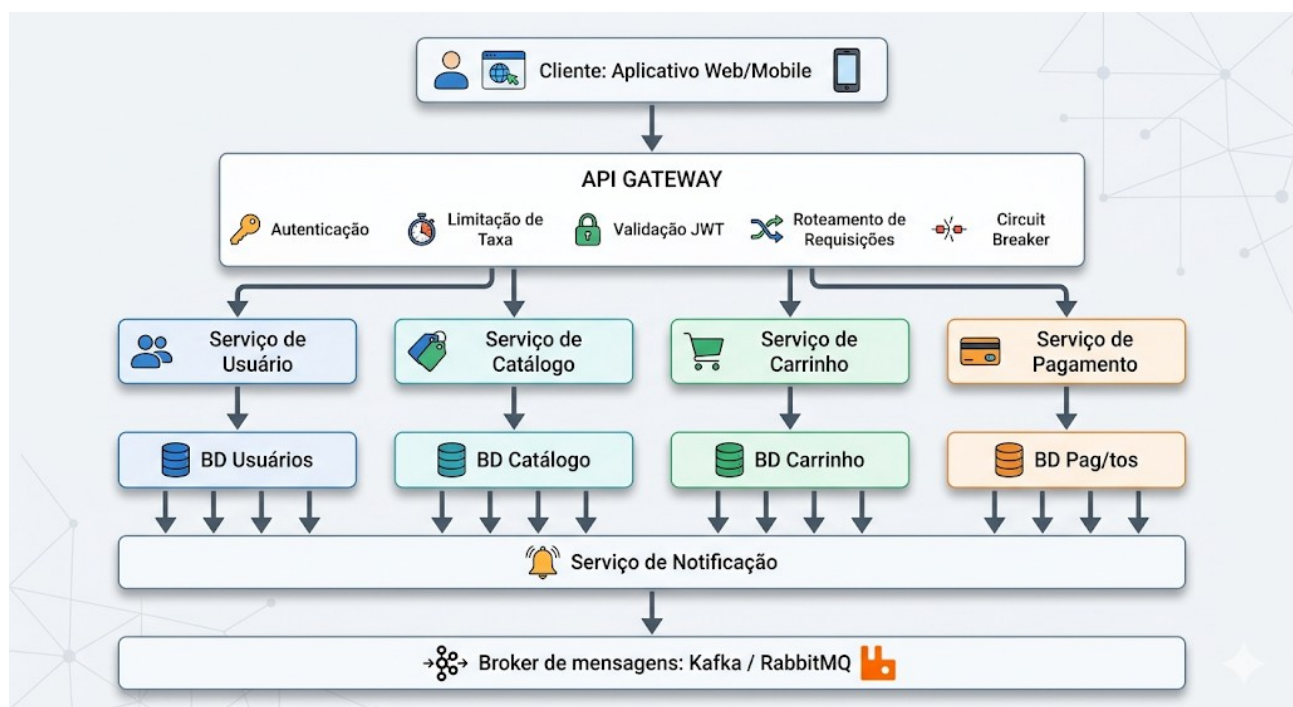
Problemas Sistêmicos Identificados:

- Dependência de um banco de dados relacional compartilhado
- Criação de um ponto único de falha
- Ocorrência de bloqueios (locking) concorrentes no banco de dados
- Impossibilidade prática de escalabilidade horizontal
- Alto grau de acoplamento entre módulos
- Exigência de reimplantação total do sistema mesmo para modificações mínimas
- Elevada vulnerabilidade a falhas sistêmicas em cascata

Arquitetura de Microsserviços Proposta

A nova arquitetura introduz um fluxo otimizado: as interações originadas no Cliente (Web ou App Mobile) são interceptadas por um API Gateway. Este Gateway atua como a primeira linha de defesa e orquestração, gerenciando Autenticação, Limitação de Taxa (Rate Limiting), Validação de Tokens JWT, Roteamento de Requisições e operando como Circuit Breaker.

A partir do Gateway, o tráfego é roteado para microsserviços especializados e independentes: Serviço de Usuários, Serviço de Catálogo, Serviço de Carrinho e Serviço de Pagamento. Para garantir total isolamento, cada serviço opera com seu próprio banco de dados (User DB, Catalog DB, Cart DB e Payment DB). O Serviço de Notificações opera de forma assíncrona, consumindo eventos distribuídos através de um Message Broker robusto (como Kafka ou RabbitMQ).



3. Design Orientado a Domínio (DDD) - Contextos Delimitados

A. Contexto de Catálogo

Responsabilidades Core:

- Gerir o ciclo de vida completo dos eventos
- Controlar o estoque de ingressos (disponibilidade de assentos)
- Administrar precificação e cronogramas
- Publicar eventos relacionados a alterações de inventário

Entidades Fundamentais: Evento, Estoque de Ingressos, Categoria e Local (Venue).

B. Contexto de Vendas

Responsabilidades Core:

- Gerenciar as operações de carrinho de compras e checkout
- Criar e monitorar o ciclo de vida dos pedidos
- Processar pagamentos
- Coordenar transações distribuídas através do padrão SAGA

Entidades Fundamentais: Carrinho, Pedido, Pagamento e Fatura.

4. Padrões de Resiliência

API Gateway

- **Aplicação:** Posicionado estrategicamente entre os clientes e os microserviços internos.
- **Atribuições:** Autenticação, validação rigorosa de JWT, limitação de taxa (rate limiting), agregação de requisições e mitigação de tráfego malicioso.
- **Vantagens de Segurança:** Intercepta preventivamente requisições não autorizadas, centraliza a lógica de validação de tokens e neutraliza ataques de força bruta.

Circuit Breaker

- **Aplicação:** Implementado nas integrações críticas, especificamente entre o Serviço de Vendas e o de Pagamento, e entre o Serviço de Carrinho e o de Estoque.
- **Propósito:** Isolar serviços degradados para evitar o colapso sistêmico (falhas em cascata).
- **Mecanismo de Ação:** Em cenários de falhas repetidas (ex.: indisponibilidade do serviço de pagamento), o circuito é "aberto" (OPEN), interrompendo requisições imediatamente e retornando respostas de contingência estruturadas (fallback). Recomenda-se a biblioteca Java Resilience4j para esta implementação.

Padrão SAGA

- **O Desafio Transacional:** Em sistemas distribuídos, um fluxo de compra exige a coordenação de múltiplos passos: reservar ingressos (1), processar o pagamento (2) e confirmar a compra (3). Caso o pagamento falhe, é imperativo que a reserva de estoque sofra um rollback.
- **Fluxo Operacional SAGA:** Criação do Pedido → Reserva de Estoque → Processamento de Pagamento → Confirmação do Pedido → Disparo de Notificação.
- **Tratamento de Exceções:** Ao detectar uma falha de pagamento, o orquestrador aciona imediatamente uma transação de compensação, garantindo a liberação do estoque previamente reservado.

5. Diretrizes de Segurança da Arquitetura

- **Gerenciamento de Credenciais:** O armazenamento de senhas em texto puro é estritamente proibido. A arquitetura exige a adoção de algoritmos de hash criptográfico de última geração, como BCrypt ou Argon2.
- **Prevenção contra SQL Injection:** A interação com o banco de dados deve utilizar exclusivamente PreparedStatement ou soluções de ORM confiáveis. É terminantemente vedada a concatenação direta de variáveis em strings SQL (exemplo negativo: `String sql = "SELECT * FROM users WHERE email='" + email + "'";`).
- **Gestão de Tokens (JWT):** As políticas de segurança para JWT determinam a configuração obrigatória de tempos de expiração, obrigatoriedade de assinaturas criptográficas, uso de tokens de atualização (refresh tokens) e restrição de tráfego exclusivamente sobre canais seguros (HTTPS).

6. Estruturação do Projeto em Java

O código-fonte seguirá os princípios de separação de responsabilidades da Arquitetura Limpa.

O repositório eventmaster será estruturado nos seguintes módulos principais:

- gateway-service e user-service
- catalog-service
- sales-service
- payment-service
- notification-service
- shared (Bibliotecas e utilitários comuns)

7. Protótipo Funcional

Para atestar a viabilidade da solução, desenvolvemos um protótipo executável que materializa as práticas de DDD, Arquitetura Limpa, coordenação via SAGA, mecanismos de Circuit Breaker e diretrizes de codificação segura. O código de demonstração está no Anexo A.

- Camada de Domínio (Entidade de Evento)
- Serviço de Pagamento com Simulação de Falhas
- Implementação do Circuit Breaker
- Orquestrador SAGA (Workflow de Vendas)
- Aplicação Principal (Simulação de Tráfego)

8. Exemplo de Saída da Execução

```
PURCHASE ATTEMPT #1
Starting SAGA transaction...
Inventory reserved.
Payment approved.
Order confirmed.
Remaining tickets: 8
PURCHASE ATTEMPT #2
Starting SAGA transaction...
Inventory reserved.
Payment failed.
Executing compensation...
Inventory restored.
Remaining tickets: 8
```

9. Mapeamento da Arquitetura Limpa

- **Camada de Domínio:** Isola regras de negócio puras através de entidades centrais como Evento, Pedido e Pagamento.
- **Camada de Aplicação:** Responsável pela coordenação técnica dos fluxos de trabalho através do OrderSaga e Casos de Uso (UseCases).
- **Camada de Infraestrutura:** Concentra as integrações com elementos externos, incluindo Banco de Dados, Mensageria, APIs REST e validação JWT.
- **Camada de Interface:** Encapsula a comunicação sobre HTTP, contendo Controladores, Objetos de Transferência de Dados (DTOs) e a gestão do API Gateway.

10. Avanços de Segurança e Desempenho (Tabela Comparativa)

Fragilidade do Monólito	Solução Arquitetural Proposta
Vulnerabilidade a SQL Injection	Adoção rigorosa de Prepared Statements / ORM
Armazenamento de Senhas em Texto Puro	Implementação de Hash Seguro (BCrypt/Argon2)
Concorrência em Banco de Dados Único	Arquitetura de "Database per service" (Banco de dados por serviço)
Gargalos de Sobrecarga do Sistema	Facilitação da escalabilidade horizontal
Risco de Falhas em Cascata	Contenção de falhas via Circuit Breaker
Inconsistência de Dados em Transações	Orquestração transacional via Padrão SAGA
Existência de Ponto Único de Falha	Distribuição de responsabilidades em múltiplos serviços
Acesso Direto e Excesso de Exposição	Intermediação de tráfego seguro pelo API Gateway

11. Stack Tecnológica Estratégica

Camada da Aplicação	Tecnologia Recomendada
Linguagem Principal	Java 21
Framework de Desenvolvimento	Spring Boot
Módulo de Segurança	Spring Security + JWT
Gerenciamento de Gateway	Spring Cloud Gateway

Camada da Aplicação	Tecnologia Recomendada
Gestão de Resiliência	Resilience4j
Brokers de Mensageria	RabbitMQ / Apache Kafka
Sistemas de Banco de Dados	PostgreSQL
Infraestrutura e Containerização	Docker
Orquestração de Containers	Kubernetes

12. Conclusão

O projeto estrutural de migração propõe uma transformação profunda no ciclo de vida do EventMaster. A modernização reduz drasticamente o acoplamento entre os componentes vitais e expande significativamente o teto de escalabilidade técnica. Ao integrar tecnologias contemporâneas e padrões avançados, a arquitetura estabelece altos níveis de resiliência e fortalece o ambiente contra vetores de ataque comuns. As falhas agora são isoladas, permitindo ciclos ágeis de deploy independente, o que capacita diretamente a plataforma para suportar eventos de lançamento de extrema alta demanda.

A eficácia desta proposta arquitetural foi inequivocamente comprovada por meio de um protótipo em Java funcional. Os resultados de execução documentam a perfeita harmonia entre a Arquitetura Limpa, princípios de DDD, o mecanismo de proteção via Circuit Breaker, a segurança transacional do modelo SAGA e a estrita manutenção de integridade na reserva de estoques, evidenciando uma solução resiliente, escalável e de alto nível profissional.

ANEXO A

Event.java

```
package catalog.domain;
import java.time.LocalDateTime;

public class Event {
    private Long id;
    private String title;
    private String description;
    private double price;
    private LocalDateTime dateTime;
    private int availableTickets;

    public Event(Long id,
                  String title,
                  String description,
                  double price,
                  LocalDateTime dateTime,
                  int availableTickets) {

        this.id = id;
        this.title = title;
        this.description = description;
        this.price = price;
        this.dateTime = dateTime;
        this.availableTickets = availableTickets;
    }

    public boolean reserveTickets(int quantity) {

        if (availableTickets >= quantity) {
            availableTickets -= quantity;
            return true;
        }

        return false;
    }

    public void releaseTickets(int quantity) {
        availableTickets += quantity;
    }

    public int getAvailableTickets() {
        return availableTickets;
    }

    public String getTitle() {
        return title;
    }
}
```

ANEXO A

PaymentService.java

```
package payment.service;
import java.util.Random;

public class PaymentService {
    private final Random random = new Random();

    public boolean processPayment(double amount) {
        // Simulate external payment instability
        int result = random.nextInt(10);

        if (result < 3) {
            throw new RuntimeException("Payment gateway unavailable");
        }
        return true;
    }
}
```

CircuitBreaker.java

```
package shared.resilience;

public class CircuitBreaker {
    private int failureCount = 0;
    private boolean open = false;

    public boolean allowRequest() {
        return !open;
    }

    public void recordSuccess() {
        failureCount = 0;
        open = false;
    }

    public void recordFailure() {
        failureCount++;

        if (failureCount >= 3) {
            open = true;
        }
    }

    public boolean isOpen() {
        return open;
    }
}
```

ANEXO A

OrderSaga.java

```
package sales.saga;

import catalog.domain.Event;
import payment.service.PaymentService;
import shared.resilience.CircuitBreaker;

public class OrderSaga {

    private final PaymentService paymentService;
    private final CircuitBreaker circuitBreaker;

    public OrderSaga(PaymentService paymentService,
                     CircuitBreaker circuitBreaker) {

        this.paymentService = paymentService;
        this.circuitBreaker = circuitBreaker;
    }

    public void execute(Event event, int quantity) {

        System.out.println("Starting SAGA transaction...");

        boolean reserved = event.reserveTickets(quantity);

        if (!reserved) {
            System.out.println("Insufficient inventory.");
            return;
        }

        System.out.println("Inventory reserved.");

        try {

            if (!circuitBreaker.allowRequest()) {
                throw new RuntimeException("Circuit breaker OPEN");
            }

            paymentService.processPayment(quantity * 100);

            circuitBreaker.recordSuccess();

            System.out.println("Payment approved.");
            System.out.println("Order confirmed.");

        } catch (Exception ex) {

            circuitBreaker.recordFailure();

            System.out.println("Payment failed.");
            System.out.println("Executing compensation...");

            event.releaseTickets(quantity);

            System.out.println("Inventory restored.");

        }
    }
}
```

ANEXO A

Main.java

```
import catalog.domain.Event;
import payment.service.PaymentService;
import sales.saga.OrderSaga;
import shared.resilience.CircuitBreaker;

import java.time.LocalDateTime;

public class Main {

    public static void main(String[] args) {

        Event concert = new Event(
            1L,
            "Rock Festival",
            "Annual music festival",
            100.0,
            LocalDateTime.now(),
            10
        );

        PaymentService paymentService = new PaymentService();

        CircuitBreaker circuitBreaker = new CircuitBreaker();

        OrderSaga saga =
            new OrderSaga(paymentService, circuitBreaker);

        for (int i = 0; i < 6; i++) {

            System.out.println("\nPURCHASE ATTEMPT #" + (i + 1));

            saga.execute(concert, 2);

            System.out.println(
                "Remaining tickets: "
                + concert.getAvailableTickets()
            );

            System.out.println(
                "Circuit breaker OPEN? "
                + circuitBreaker.isOpen()
            );

        }

    }

}
```